
Gmail Spam Detection Using Logistic Regression

Abstract:

This project focuses on developing a machine learning-based spam detection system using Logistic Regression. With the rise in unwanted and potentially harmful emails, automatic spam detection has become a necessity for ensuring a safe and efficient communication experience.

By leveraging natural language processing techniques and supervised learning, the system classifies emails as either spam or non-spam with high accuracy. The model is trained on a real-world dataset and validated using standard evaluation metrics. This project demonstrates the effectiveness of Logistic Regression in text classification tasks, particularly in detecting email spam with minimal false detections.

Objective:

The primary objective of this project is to develop a machine learning model capable of classifying Gmail messages as either spam or legitimate (ham). Email spam poses security risks and disrupts user experience, making it essential to build automated filtering systems.

This project aims to solve the classification problem using Logistic Regression, a widely-used algorithm for binary classification tasks. By applying this model, we strive to create a system that accurately detects unwanted or harmful emails and improves the overall email handling experience.

Introduction:

Email remains one of the most widely used communication platforms in both personal and professional settings. However, it is also one of the primary targets for spam messages, which can include unwanted advertisements, phishing attempts, and malicious content. These spam emails not only clutter inboxes but can also pose serious security threats to users.

To address this problem, machine learning techniques offer automated and efficient solutions for identifying spam emails. In this project, we aim to build a spam detection system using Logistic Regression, a popular algorithm for binary classification. The model is trained on a labeled dataset of email messages and is designed to classify new emails as either spam or non-spam based on their content.

This project highlights how natural language processing (NLP) and machine learning can be combined to solve real-world problems effectively, with the potential for integration into larger email systems for enhanced filtering.

Methodology:

The methodology followed in this project involves several key stages:

1. Data Collection: A publicly available dataset containing labeled spam and non-spam email messages is used. The dataset includes message content and binary labels indicating spam status.

2. Text Preprocessing: Text data is cleaned by removing special characters, numbers, and stop words. Tokenization and lowercasing are applied to normalize the text.

3. Feature Extraction: TF-IDF (Term Frequency-Inverse Document Frequency) vectorization is used to convert the text into numerical format suitable for machine learning models.

4. Model Selection: Logistic Regression is chosen due to its effectiveness and simplicity for binary classification problems.

5. Training and Testing: The dataset is split into training and testing sets. The model is trained on the training set and evaluated using the test set.

6. Evaluation: Performance is measured using accuracy, precision, recall, F1-score, and confusion matrix.

Gmail spam-detection-logistic-regression code:

The following implementation demonstrates a complete workflow for fruit image classification, including data preprocessing, model development using EfficientNetB0, and evaluation with visualization of results.

```
[1]: import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix
```

```
[2]: #dataset is separated by tab, so we use seperator='\t'
data = pd.read_csv('../input/sms-data-labelled-spam-and-non-spam/
.SMSSpamCollection', sep='\t', names=['label', 'message'])
```

0.0.1 Label:

Spam: message is spam **ham:** message is not spam

```
[3]: data.head()
```

```
[3]:   label      message
0    ham  Go until jurong point, crazy.. Available only ...
1    ham              Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3    ham  U dun say so early hor... U c already then say...
4    ham  Nah I don't think he goes to usf, he lives aro...
```

```
[4]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
label    5572  non-null  object
message  5572  non-null  object
dtypes: object(2)
memory usage: 87.2+ KB
```

convert our labels to binary variables, 0 to represent 'ham'(i.e. not spam) and 1 to represent 'spam'

```
[5]: #use '1' for spam and '0' for not spam
data['label'] = data.label.map({'ham':0, 'spam':1})
data.head()
```

```
[5]:   label      message
0      0  Go until jurong point, crazy.. Available only ...
1      0                Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...
```

```
[6]: # split into training and testing sets
# USE from sklearn.model_selection import train_test_split to avoid seeing_
# deprecation warning.
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(data['message'],
                                                    data['label'],
                                                    test_size =0.2,
                                                    random_state=1)

print('Number of rows in the total set: {}'.format(data.shape[0]))
print('Number of rows in the training set: {}'.format(X_train.shape[0]))
print('Number of rows in the test set: {}'.format(X_test.shape[0]))
```

```
Number of rows in the total set: 5572
Number of rows in the training set: 4457
Number of rows in the test set: 1115
```

0.0.2 Frequency distribution

Our objective here is to convert this set of text to a frequency distribution matrix

Note - The CountVectorizer method automatically converts all tokenized words to their lower case form so that it does not treat words like 'He' and 'he' differently. It does this using the lowercase parameter which is by default set to True.

- It also ignores all punctuation so that words followed by a punctuation mark (for example: 'hello!') are not treated differently than the same words not prefixed or suffixed by a punctuation mark (for example: 'hello').

```
[7]: from sklearn.feature_extraction.text import CountVectorizer
count_vector = CountVectorizer()
```

fit_transform() Learn the vocabulary dictionary and return term-document matrix. #####
transform() Transform documents to document-term matrix.

```
[8]: # Fit the training data and then return the matrix
training_data = count_vector.fit_transform(X_train).toarray()

# Transform testing data and return the matrix. Note we are not fitting the
# testing data into the CountVectorizer()
testing_data = count_vector.transform(X_test).toarray()
```

```
[9]: frequency_matrix = pd.DataFrame(training_data,
                                     columns = count_vector.get_feature_names())
frequency_matrix.head()
```

```
[9]: 00 000 008704050406 0121 01223585236 01223585334 0125698789 02 \
0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0
2 0 0 0 0 0 0 0 0 0
3 0 0 0 0 0 0 0 0 0
4 0 0 0 0 0 0 0 0 0

0207 02072069400 ... zed zeros zhong zindgi zoe zoom zouk zyada \
0 0 0 ... 0 0 0 0 0 0 0 0
1 0 0 ... 0 0 0 0 0 0 0 0
2 0 0 ... 0 0 0 0 0 0 0 0
3 0 0 ... 0 0 0 0 0 0 0 0
4 0 0 ... 0 0 0 0 0 0 0 0

èn ud
0 0 0
1 0 0
2 0 0
3 0 0
4 0 0

[5 rows x 7714 columns]
```

```
[10]: testing_data
```

```
[10]: array([[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]])
```

```
[11]: #Train the data
clf = LogisticRegression(random_state=0).fit(training_data, y_train)
```

```
[ #predict the value
predictions = clf.predict(testing_data)
```

```
[ predictions
```

```
[13]: array([0, 0, 0, ..., 0, 0, 0])
```

0.0.3 Result

```
[ print('Accuracy score: ', format(accuracy_score(y_test, predictions)))
  print('Precision score: ', format(precision_score(y_test, predictions)))
  print('Recall score: ', format(recall_score(y_test, predictions)))
  print('F1 score: ', format(f1_score(y_test, predictions)))
  print('\nConfusion Matrix :\n', confusion_matrix(y_test, predictions))
```

```
Accuracy score: 0.989237668161435
Precision score: 0.9927007299270073
Recall score: 0.9251700680272109
F1 score: 0.9577464788732395
```

```
Confusion
Matrix :
[[967  1]
 [ 11 136]]
```

Conclusion:

This methodology ensures a robust approach for building a reliable spam detection model. The Gmail Spam Detection project illustrates how machine learning can be used to solve real-world text classification problems. Logistic Regression proved to be a strong baseline algorithm, offering high accuracy, interpretability, and fast training time.

The model achieved a high F1-score and excellent ROC-AUC score, making it suitable for real-world deployment. With further optimization or inclusion of ensemble models, the system could potentially outperform more complex solutions. This project demonstrates the practical application of NLP and ML to improve email security and user experience.